Introduction to Artificial Intelligence

CS188 6 - Part II: Game Tree Pruning

Namyup Kim

Soongsil University

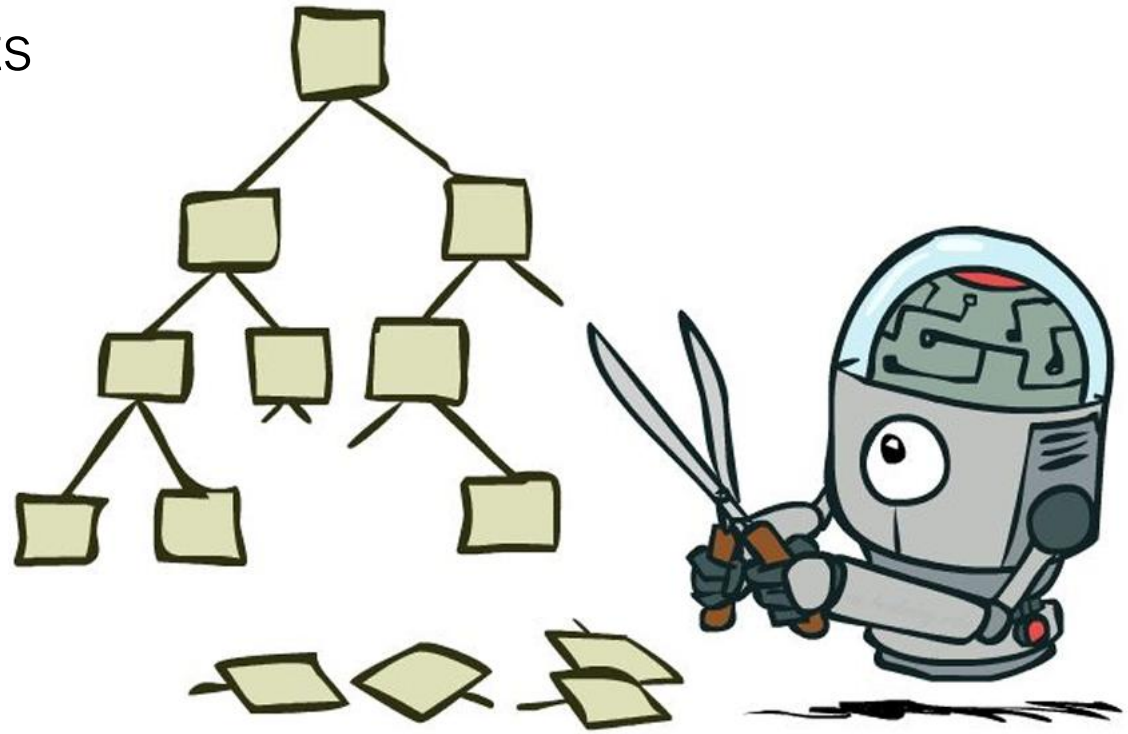


Source: UC Berkeley CS188, slides by Dan Klein and Pieter Abbeel

3. Game Tree Pruning

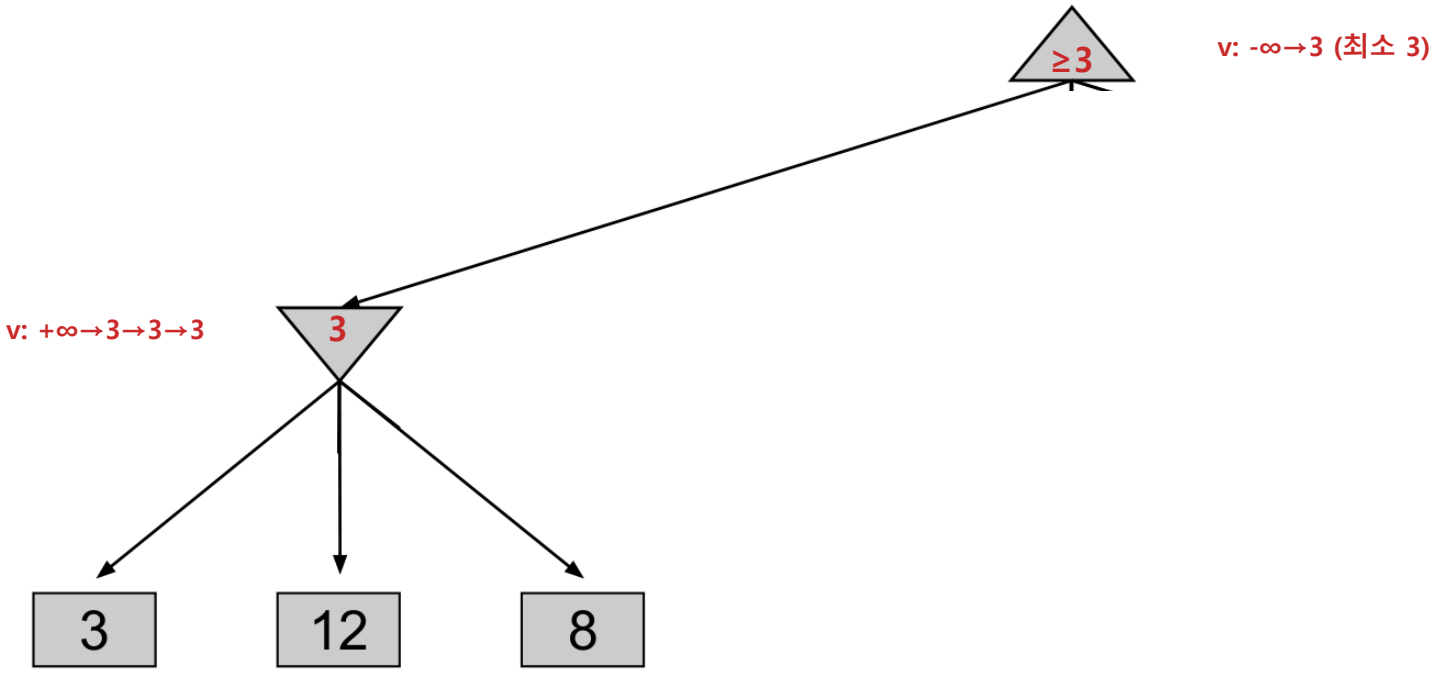
Getting the same answer without looking at the whole tree

- Working the example again, watching the bounds
- The general rule, and the two numbers **alpha** and **beta**
- What pruning changes, and one trap it sets
- Two quizzes



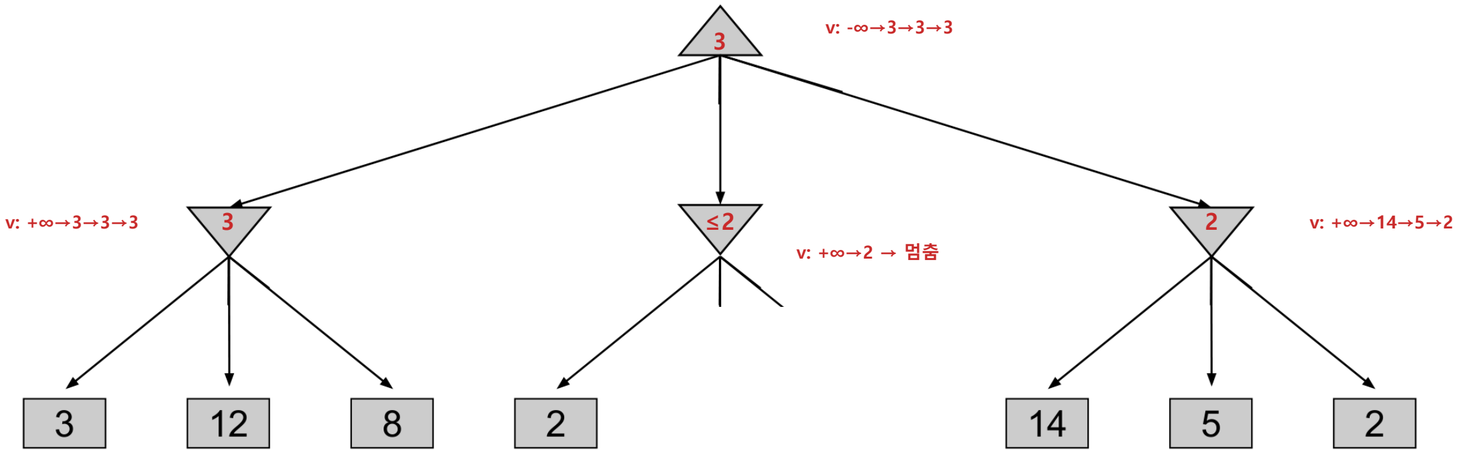
Minimax Example

- The first MIN node sees 3, then 12, then 8, and settles on 3.
- The root now knows it can get at least 3. That is a lower bound, and it will never go down.
- Keep that number in mind as we move to the second branch.



Minimax Pruning

- Move to the second branch. Its first child is 2, so this MIN node is at most 2
- The root already has 3. A branch worth at most 2 can never beat it [1]
- So, we stop. We do not look at the rest of that branch at all
- Notice what we gave up: we never learn the real value of that node. We only learn that it does not matter

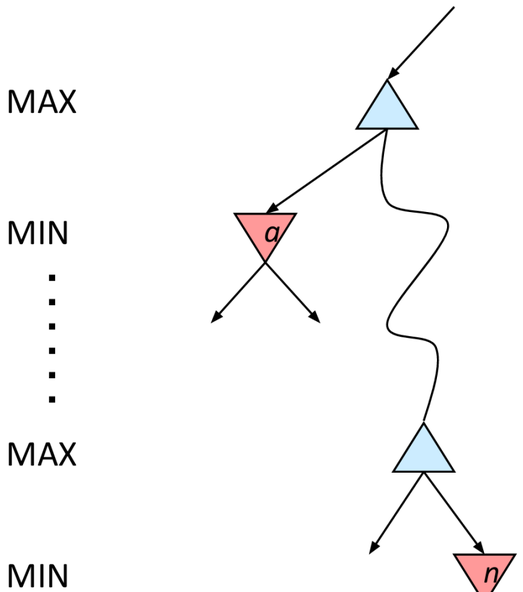


[1] **중요**: 루트가 이미 3을 확보했을 때, **최대 2인 가지는 절대 3을 이길 수 없는 이유**를 **설명할 수 있어야 함**

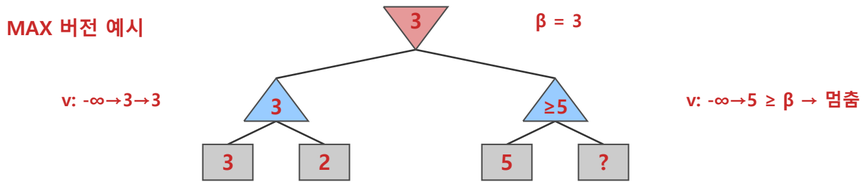
- 이유 ①: 두 번째 MIN 노드는 첫 자식에서 2를 봤으므로 값이 **최대 2(at most 2)**. MIN은 더 작은 값만 고르니, 나머지 자식을 봐도 값은 **내려갈 수만 있고 올라가지 않음**
- 이유 ②: 루트(MAX)는 이미 첫 가지에서 **3을 확보(lower bound)**. max(3, 2 이하) = 항상 3
- 그래서 나머지를 안 봐도 됨 → **가지치기(Pruning)**. 코드로는 MIN 쪽 if v ≤ α return v (α = 3)
- 대조: **MAX 노드 + MIN 부모** 경우(β 쪽). MIN 부모가 이미 2를 확보(β = 2)했는데 MAX 노드가 첫 자식에서 5를 보면 값이 **최소 5(at least 5)** → MIN 부모는 절대 안 고름 → 가지치기. 코드로는 if v ≥ β return v
- 예: 이 슬라이드에서 루트 = max(3, 2 이하, 2) = **3**

Alpha-Beta Pruning

- General configuration (MIN version)
  - We're computing the MIN-VALUE at some node n
  - We're looping over n's children
  - n's estimate of the children's min is dropping its value
  - Who cares about n's value? MAX
  - Let ALPHA α be the best value that MAX can get *at any choice point on the path to the root*
  - As soon as v drops below alpha, this node can never be chosen. **Stop and return**



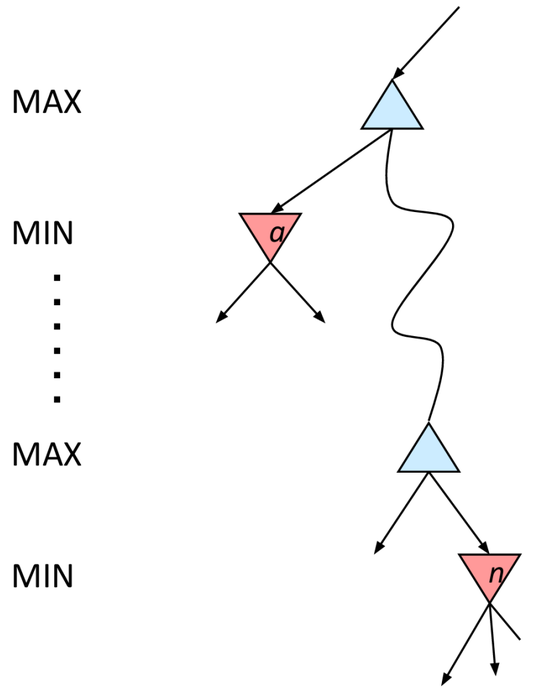
- BETA β is symmetric [4]



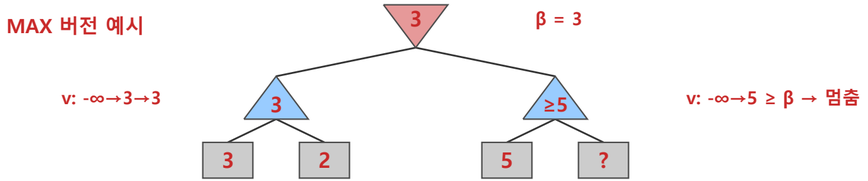
[4] **시험은 MAX 버전으로 나옴** (슬라이드는 MIN 버전)
- MAX 버전: 어떤 노드 n에서 **MAX-VALUE**를 계산, n의 자식들을 돌며 v는 **올라가기만 함**
- n의 값에 관심 있는 쪽은 **MIN**
- **BETA β** = 루트까지 가는 경로 위에서 **MIN이 이미 확보한 가장 좋은(가장 작은) 값**
- v가 β 이상이 되는 순간(**v ≥ β**) n은 절대 선택되지 않음 → **멈추고 반환(Stop and return)**. 코드: if v ≥ β return v
- 이유: v는 더 커질 수만 있는데 MIN은 이미 β 이하를 가질 수 있으니 이 가지를 고를 일이 없음
- 대조(MIN 버전, 슬라이드): v가 **α 이하**로 떨어지면 멈춤, 관심 있는 쪽은 MAX
- 예(왼쪽 아래 그림): 루트 MIN이 첫 가지에서 3 확보(β = 3) → 둘째 MAX가 첫 자식 5를 보면 v ≥ 5 ≥ 3 → 나머지(?) 안 봄. 루트 = min(3, 5 이상) = **3**

Alpha-Beta Pruning

- General configuration (MIN version)
  - We're computing the MIN-VALUE at some node n
  - We're looping over n's children
  - n's estimate of the children's min is dropping its value
  - Who cares about n's value? MAX
  - Let ALPHA α be the best value that MAX can get *at any choice point on the path to the root*
  - As soon as v drops below alpha, this node can never be chosen. **Stop and return**



- BETA β is symmetric



7 Q. α, β는 미리 정해 둔 값? → **아님. 탐색하면서 채워지는 값**

- 시작: 루트를 max-value(루트, **α = -∞, β = +∞**)로 호출 → 아직 아무것도 확보 못 했다는 뜻
- MAX 노드는 자식 값을 볼 때마다 **α를 올리고**, MIN 노드는 **β를 내림** (p6 ⑤번 줄)
- 자식을 부를 때 지금의 α, β를 **인자로 아래로 넘김**. 자식이 바꾼 α, β는 부모에게 안 돌아가고 **값 v만 돌아감**
- 대조: 리프 값(3, 12, 8…)은 문제에 **미리 주어진 값**, α·β는 **계산 중에 생기는 값**
- 예(p4 트리): 루트 α = -∞ → 첫 MIN이 3 반환 → α = **3** → 둘째 MIN은 α = 3을 받아서 2를 보자마자 가지치기

Alpha-Beta Implementation

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
① initialize v = -∞
② for each successor of state:
③       v = max(v, value(successor, α, β))
④       if v ≥ β return v
⑤       α = max(α, v)
⑥ return v
```

```
def min-value(state , α, β):
① initialize v = +∞
② for each successor of state:
③       v = min(v, value(successor, α, β))
④       if v ≤ α return v
⑤       β = min(β, v)
⑥ return v
```

Early stop, Prune!

Update the best MIN

2 MAX의 초기값(initial value) v는 **-∞(음의 무한대)**
- 이유: MAX는 자식 값 중 **가장 큰 값**을 고름. 어떤 값보다도 작은 -∞에서 시작하면 **첫 자식 값이 무조건 v를 대신**하고, 이후 더 큰 값이 나올 때만 바뀜 → 초기값이 결과를 왜곡하지 않음
- 대조: MIN은 +∞에서 시작 (3번 필기)
- 예: 자식 3, 12, 8 → v = -∞ → 3 → 12 → 12, 결과 **12**
- 만약 0에서 시작하면 자식이 전부 음수(-5, -2)일 때 답이 -2가 아니라 **0으로 틀림**

3 MIN의 초기값(initial value) v는 **+∞(양의 무한대)**
- 이유: MIN은 자식 값 중 **가장 작은 값**을 고름. 어떤 값보다도 큰 +∞에서 시작하면 **첫 자식 값이 무조건 v를 대신**하고, 이후 더 작은 값이 나올 때만 바뀜
- 대조: MAX는 -∞에서 시작 (2번 필기) → 서로 **대칭**
- 예: 자식 3, 12, 8 → v = +∞ → 3 → 3 → 3, 결과 **3** (p3의 첫 MIN 노드)

Alpha-Beta Implementation

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
① initialize v = -∞
② for each successor of state:
③       v = max(v, value(successor, α, β))
④       if v ≥ β return v
⑤       α = max(α, v)
⑥ return v
```

```
def min-value(state , α, β):
① initialize v = +∞
② for each successor of state:
③       v = min(v, value(successor, α, β))
④       if v ≤ α return v        Early stop, Prune!
⑤       β = min(β, v)            Update the best MIN
⑥ return v
```

5 코드 한 줄씩 (왼쪽 번호 = 코드 줄)

- **α**: 루트까지 경로에서 MAX가 확보한 최선값(이보다 낮으면 MAX가 안 고름), **β**: MIN이 확보한 최선값(이보다 높으면 MIN이 안 고름)
- ① v 초기화: MAX는 -∞, MIN은 +∞ (2·3번 필기)
- ② 자식(successor)을 하나씩 봄. value()는 자식 차례에 맞게 max-value/min-value를 **재귀 호출**하고 α, β를 그대로 넘김
- ③ v 갱신: MAX는 더 큰 값, MIN은 더 작은 값으로
- ④ **가지치기 검사**: MAX는 **v ≥ β**, MIN은 **v ≤ α**면 나머지 자식을 안 보고 바로 반환 (Early stop, Prune!)
- ⑤ **경계 갱신**: MAX는 α = max(α, v), MIN은 β = min(β, v) (Update the best MIN) → 다음 자식에게 더 좁은 범위를 넘김
- 순서 중요: ④ 검사 먼저, ⑤ 갱신 나중
- ⑥ 다 봤으면 v 반환
- 예(p4 트리): 루트 max-value(α=-∞, β=+∞)

첫 MIN: 3, 12, 8 → v = **3** 반환 → 루트 α = **3**

둘째 MIN(α=3): 2 → v = 2 ≤ α=3 → **④에서 바로 반환(가지치기)**

셋째 MIN(α=3): 14 → 5 → 2, v = 2 ≤ 3 → 반환

루트 = max(3, 2, 2) = **3**

Alpha-Beta Implementation

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
① initialize v = -∞
② for each successor of state:
③       v = max(v, value(successor, α, β))
④       if v ≥ β return v
⑤       α = max(α, v)
⑥ return v
```

```
def min-value(state , α, β):
① initialize v = +∞
② for each successor of state:
③       v = min(v, value(successor, α, β))
④       if v ≤ α return v
⑤       β = min(β, v)
⑥ return v
```

Early stop, Prune!

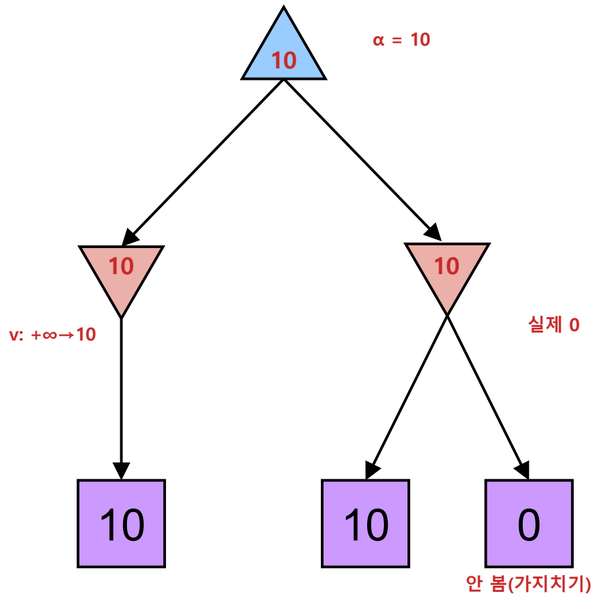
Update the best MIN

11 **중요**: ④에서 **return하면 그 자리에서 for 반복(looping)이 끝남** → 남은 자식은 안 봄 = 이게 **가지치기**

- 이유: return은 함수 자체를 끝내고 v를 부모에게 돌려줌 → for문도 같이 끝나고 ⑤ 갱신도 실행 안 됨
- 대조: ④ 조건이 거짓이면 return 안 함 → ⑤ α/β 갱신 후 **다음 자식으로 계속**
- 예(Quiz 2 e): f = 100 → 100 ≥ β = 10 → return 100 → for가 끝나 **g는 아예 안 봄**

Alpha-Beta Pruning Properties

- Pruning has NO effect on the minimax value computed for the root. The answer is exactly the same! 6
- But the values of intermediate nodes may be **WRONG**, because we stopped early and returned a bound instead of a value.
- So, the naive version does not let you do the best *Action Selection* off the root. You need correctness one level down.
- Good child ordering matters enormously. With perfect ordering the time goes from $O(b^m)$ to $O(b^{\frac{m}{2}})$
- This is a simple example of metareasoning (computing about what to compute)



6 **중요**: 가지치기(Pruning)는 **루트의 minimax 값에 영향 없음** → 안 자른 것과 **답이 똑같음**

• 이유: 자르는 곳은 v ≤ α(MIN) 또는 v ≥ β(MAX)라서 **루트가 절대 고르지 않는다고 확정된 가지**뿐. 그 안의 값이 뭐든 루트 답은 안 바뀜

• 대조: 루트 값은 정확하지만 **중간 노드 값은 틀릴 수 있음 (WRONG)** → 진짜 값 대신 경계(bound)를 반환했기 때문

• 예(오른쪽 트리): 왼쪽 MIN = 10 → 루트 α = 10

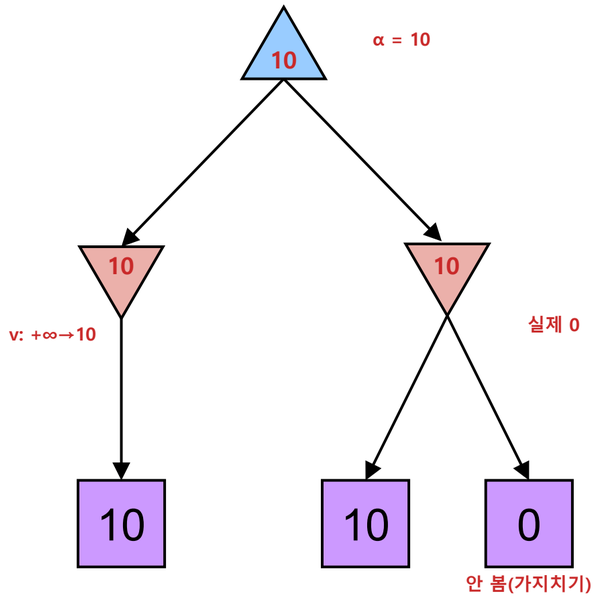
오른쪽 MIN: 첫 자식 10 → v = 10 ≤ α = 10 → **0은 안 보고 10 반환(가지치기)**

루트 = max(10, 10) = **10**, 다 봐도 max(10, 0) = **10** → 같음

하지만 오른쪽 MIN의 진짜 값은 **0**인데 10으로 나옴 → 루트에서 두 가지가 똑같이 10으로 보여 **오른쪽을 고르면 틀린 행동**(Action Selection 문제)

Alpha-Beta Pruning Properties

- Pruning has NO effect on the minimax value computed for the root. The answer is exactly the same!
- But the values of intermediate nodes may be **WRONG**, because we stopped early and returned a bound instead of a value.
- So, the naive version does not let you do the best ***Action Selection*** off the root. You need correctness one level down.
- Good child ordering matters enormously.[8] With perfect ordering the time goes from $O(b^m)$ to $O(b^{\frac{m}{2}})$
- This is a simple example of metareasoning (computing about what to compute)

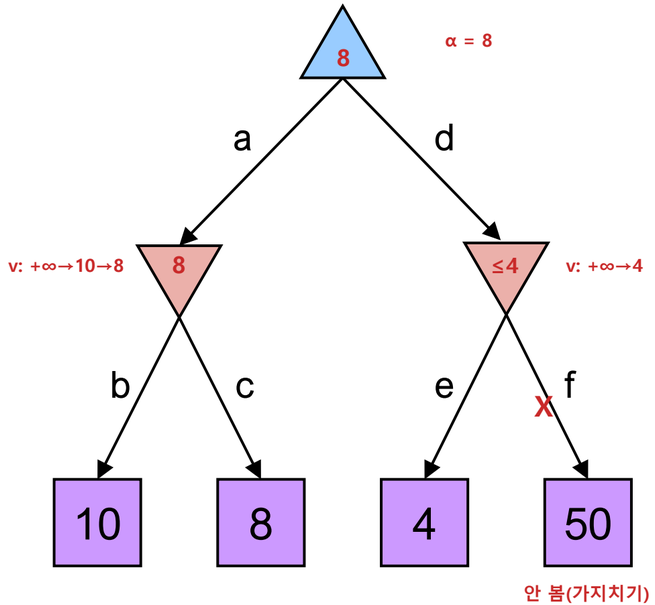


[8] **자식 순서(child ordering)가 매우 중요**: 완벽한 순서면 시간이 O(b^m) → **O(b^(m/2))**, 지수가 **1/2로 줄어드는 게 중요**

- 주의: 시간이 절반이 되는 게 아니라 **지수가 절반** = 탐색량이 **제곱근**으로 줄어듦
- 이유(체스 비유): 가지치기(Prune)로 버리는 수가 많아지니 **같은 시간에 더 많은 수(더 깊이)를 볼 수 있음** → 같은 시간에 깊이 약 **2배**
- 예: 체스 b ≈ 35, 4수 앞 → 그냥 35^4 ≈ 150만 개, 완벽한 순서면 35^2 ≈ 1,225개. 150만 개를 쓸 수 있으면 **8수 앞**까지 봄 (p11: α-β로 깊이 약 8)
- 대조: 순서가 나쁘면(나쁜 자식부터) 거의 못 자름 → minimax와 같은 **O(b^m)**
- 좋은 순서 = MAX는 **큰 값부터**, MIN은 **작은 값부터** 봄
- 예(p4 셋째 MIN): 14, 5, 2 순서라 셋 다 봄. **2를 먼저** 봤으면 2 ≤ α = 3 → 14, 5는 **바로 가지치기**

Alpha-Beta Quiz 1

- Work out the minimax value of the root
- Then: visiting left to right, which branches[9] can alpha-beta prune?

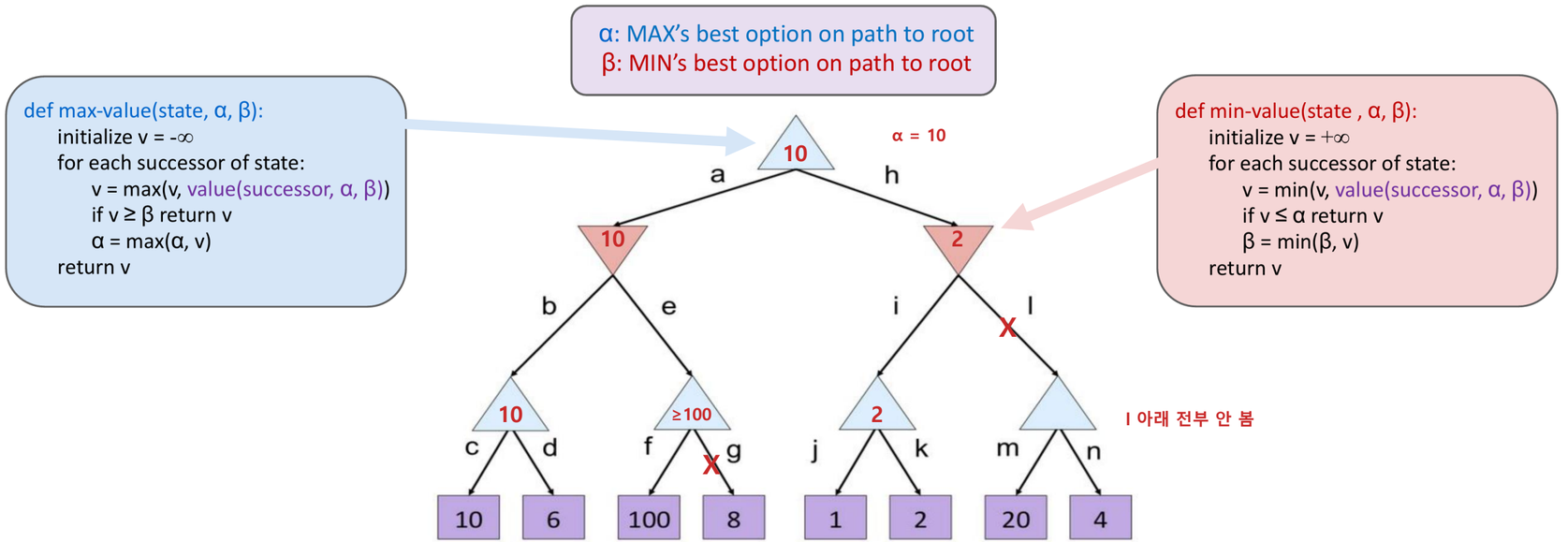


9 Quiz 1 굉장히 중요

- 답: 루트 = **8**, 가지치기 = **f**
- 풀이: 왼쪽 MIN: 10 → 8, v = 8 → 루트 α = **8**
  오른쪽 MIN(α = 8): e = 4 → v = 4 **≤ α = 8 → f(50)는 안 봄**
- 이유: 오른쪽 MIN은 4 이하인데 루트는 이미 8 확보 → 50이 뭐든 루트 답 안 바뀜 (p4와 같은 구조, MIN 버전 가지치기)
- 대조: 순서가 f(50)부터였다면 v = 50 → 50 ≤ 8 아님 → e도 봐야 해서 **가지치기 없음** (자식 순서, p7)
- 함정: 50이 커 보여도 MIN은 작은 값을 고르므로 상관없음

Alpha-Beta Quiz 2

- This one is harder to do in your head, so write the bounds down.
- Watch for the configuration where you already know the value of an earlier branch - that is where the pruning comes from



10 Quiz 2 굉장히 중요 (시험 스타일: 코드 + 트리)

- 답: 루트 = **10**, 가지치기 = **g, l**(l 아래 m, n 전부)
- 풀이 (α, β)를 적으며:

루트 max(α = -∞, β = +∞)
  a: min(-∞, +∞)
    b: max(-∞, +∞): c = 10 → α = 10, d = 6 → **10** 반환
    a의 β = **10**
    e: max(-∞, β = 10): f = 100 → v = 100 **≥ β = 10 → g 안 봄**, 100 반환
  a = min(10, 100) = **10** → 루트 α = **10**
  h: min(α = 10, +∞)
    i: max(10, +∞): j = 1, k = 2 → **2** 반환
  h: v = 2 **≤ α = 10 → l 안 봄**, 2 반환
루트 = max(10, 2) = **10**

- 이유 g: a(MIN)는 이미 10 확보, e는 100 이상 → a가 e를 안 고름
- 이유 l: 루트(MAX)는 이미 10 확보, h는 2 이하 → 루트가 h를 안 고름
- 대조: g는 **MAX 버전(v ≥ β)**, l은 **MIN 버전(v ≤ α)** → 한 문제에 둘 다 나옴

Alpha-Beta Quiz 2

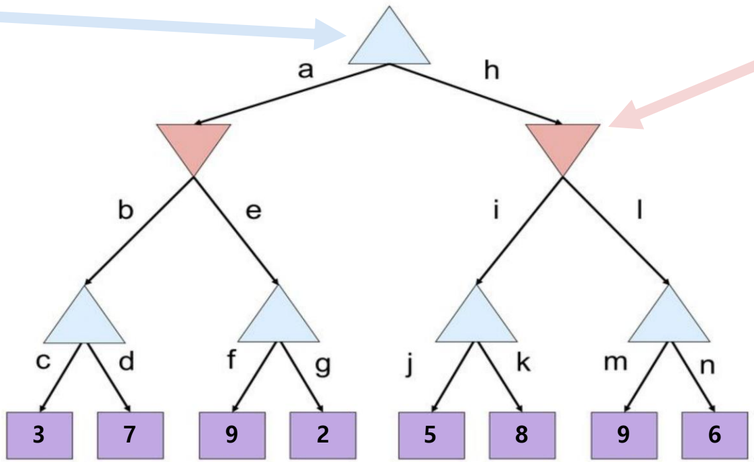
연습 문제: Quiz 2 숫자 바꿈

슬라이드의 코드대로 왼쪽 → 오른쪽 순서로 풀기 (풀이는 다음 쪽)

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

```
def min-value(state , α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```



연습 문제 1. 루트의 minimax 값은?
2. 왼쪽 → 오른쪽으로 볼 때 가지치기되는 가지는? (c~n 기호로)
3. e에서 max-value가 호출될 때 (α, β)는? 코드 몇 번째 줄에서 멈추고, 무엇을 반환하나?
4. l에서 max-value가 호출될 때 (α, β)는? 무엇을 반환하나?
5. i에서 j = 5를 본 뒤 α는?
6. h의 min-value가 끝날 때 β는? 루트는 a, h 중 어느 쪽을 고르나?
7. 이 문제의 가지치기는 MAX 버전(v ≥ β)인가, MIN 버전(v ≤ α)인가? 원래 Quiz 2와 비교하면?

코드 줄 번호 (p6과 같음): ① v 초기화 ② for 자식 ③ v 갱신 ④ 가지치기 검사 ⑤ α/β 갱신 ⑥ return

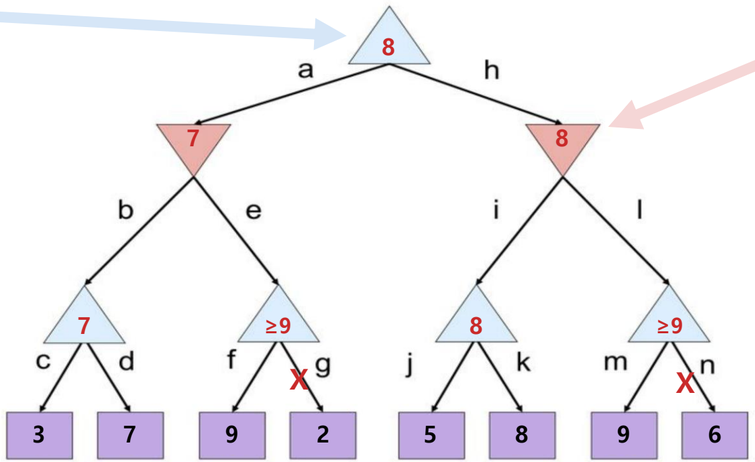
Alpha-Beta Quiz 2

연습 문제 풀이: 루트 = 8, 가지치기 = g, n

α: MAX's best option on path to root
β: MIN's best option on path to root

```
def max-value(state, α, β):
    initialize v = -∞
    for each successor of state:
        v = max(v, value(successor, α, β))
        if v ≥ β return v
        α = max(α, v)
    return v
```

```
def min-value(state , α, β):
    initialize v = +∞
    for each successor of state:
        v = min(v, value(successor, α, β))
        if v ≤ α return v
        β = min(β, v)
    return v
```



연습 문제 풀이 • 1. 루트 = **8**
• 2. **g, n**
• 3. e: (α = -∞, β = **7**) → f = 9 → 9 ≥ 7 → **④ if v ≥ β return v**에서 멈춤, **9** 반환
• 4. l: (α = **7**, β = **8**) → m = 9 → 9 ≥ 8 → **9** 반환 (n 안 봄)
• 5. α = max(7, 5) = **7 그대로** (α는 내려가지 않음, 함정)
• 6. β = **8**, 루트는 **h** (8 > 7)
• 7. 둘 다 **MAX 버전(v ≥ β)**. 원래 Quiz 2는 g(MAX 버전) + l(MIN 버전)
• 풀이:

루트 max(-∞, +∞)
  a: min(-∞, +∞)
    b: max(-∞, +∞): 3 → α = 3, 7 → **7** 반환 → a의 β = **7**
    e: max(-∞, 7): 9 **≥ 7** → g 안 봄, 9 반환
  a = min(7, 9) = **7** → 루트 α = **7**
  h: min(7, +∞)
    i: max(7, +∞): 5 → α = 7, 8 → α = 8, **8** 반환
  h: 8 ≤ 7 아님 → β = **8**
    l: max(7, 8): 9 **≥ 8** → n 안 봄, 9 반환
  h = min(8, 9) = **8**
루트 = max(7, 8) = **8**

4. Resource Limits

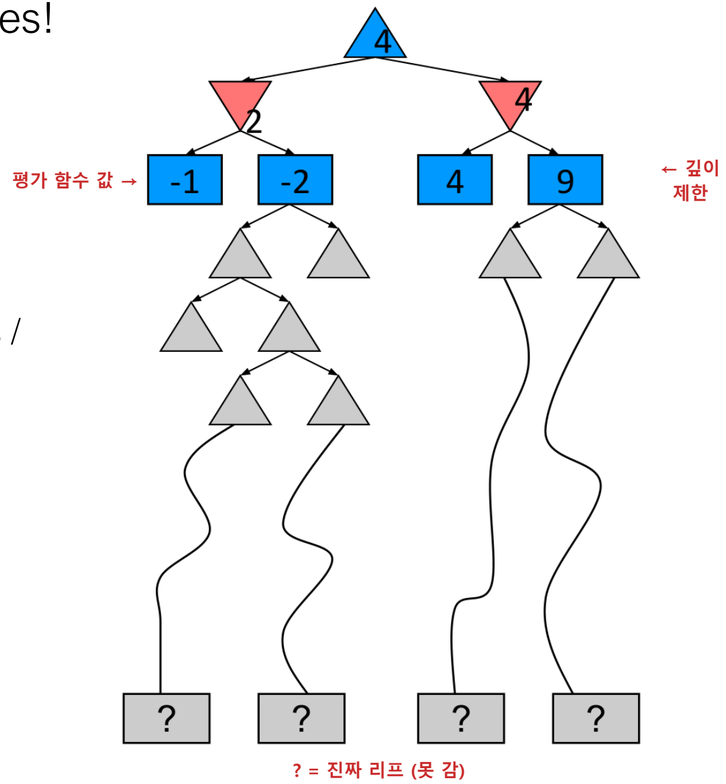
When you cannot reach the leaves, you have to invent them

- Depth-limited search, and why it needs a base case
- Replacing terminal utilities with an evaluation function
- Why a replanning Pac-Man can starve next to a dot



Resource Limits

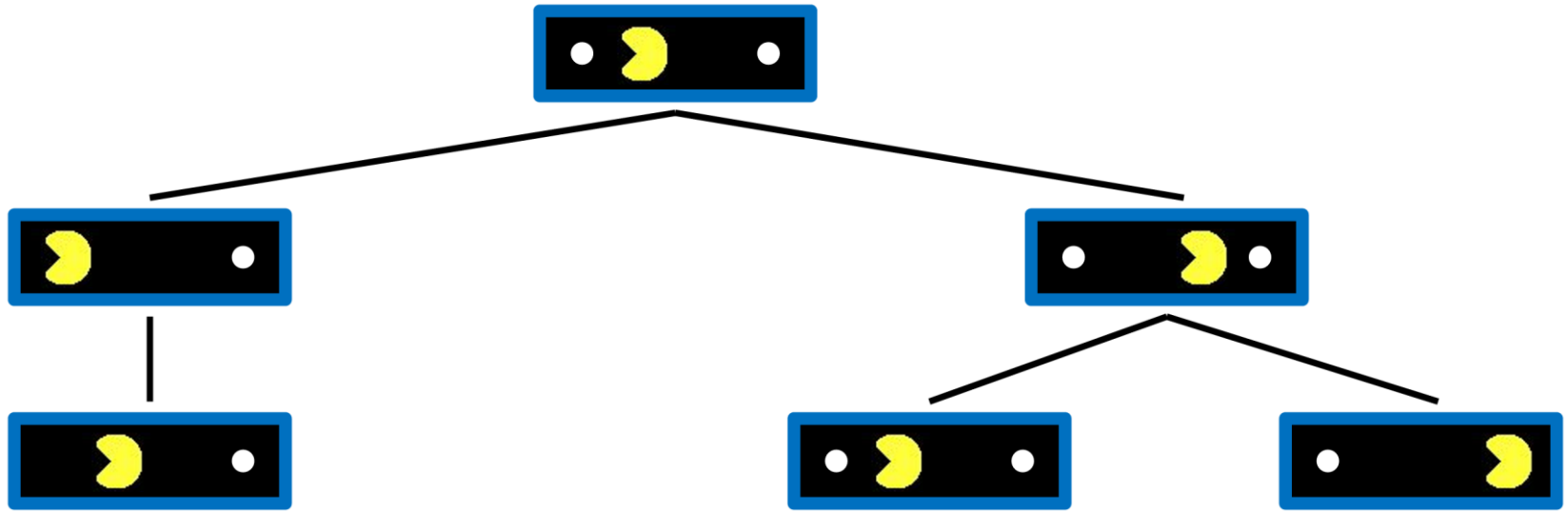
- Problem: In realistic games, cannot search to leaves!
- Solution: Depth-limited search [12]
  - Instead, search only to a limited depth in the tree
  - Replace terminal utilities with an evaluation function [13] for non-terminal positions
- Example:
  - Suppose we have 100 seconds, can explore 10K nodes / sec
  - So can check 1M nodes per move
  - α-β reaches about depth 8 – decent chess program
- Not guarantee of optimal play
- More plies makes a BIG difference
- Use iterative deepening for an anytime algorithm



12 깊이 제한 탐색(Depth-limited search): 실제 게임은 끝(리프)까지 못 가니 **정해진 깊이까지만** 트리를 보고 멈춤
- 이유: 100초 × 초당 1만 노드 = 한 수에 **100만 노드**만 볼 수 있음 → α-β로 깊이 약 8 (슬라이드)
- 대조: 원래 minimax/α-β는 **리프(게임 끝)까지** 봐서 최적 보장. 깊이 제한은 **최적 보장 없음(Not guarantee of optimal play)**
- 깊이(plies)가 깊을수록 훨씬 좋아짐 → 시간이 되는 만큼 깊이 1, 2, 3...으로 늘려 가는 **iterative deepening**: 언제 멈춰도 답이 있음(anytime)
- 예(오른쪽 트리): 깊이 2에서 멈춤 → 그 아래 **?(진짜 리프)는 못 감**

13 평가 함수(evaluation function): 깊이 제한으로 멈춘 곳(**게임이 안 끝난 위치, non-terminal**)에 점수를 매기는 함수 → 원래 리프의 효용(terminal utility) 자리를 대신함
- 이상적: 그 위치의 **진짜 minimax 값**을 돌려줌 / 실제: **근사값**, 특징 가중합 $Eval(s) = w_1f_1(s) + w_2f_2(s) + \dots$ (p14)
- 대조: terminal utility = 게임이 **끝났을 때 정확한 점수**(이김/짐), 평가 함수 = **안 끝났을 때 추정 점수**
- 예(오른쪽 트리): 파란 네모 -1, -2, 4, 9가 평가 함수 점수 → MIN: min(-1, -2) = **-2**, min(4, 9) = **4** → 루트 MAX: max(-2, 4) = **4**
- 예(체스, p14): $f_1(s)$ = 흰 퀸 수 − 검은 퀸 수
- 함정: 평가 점수가 '지금 먹기'와 '나중에 먹기'를 똑같이 보면 계속 미룸 → p12 팩맨이 점 옆에서 굶음

Why Pacman Starves

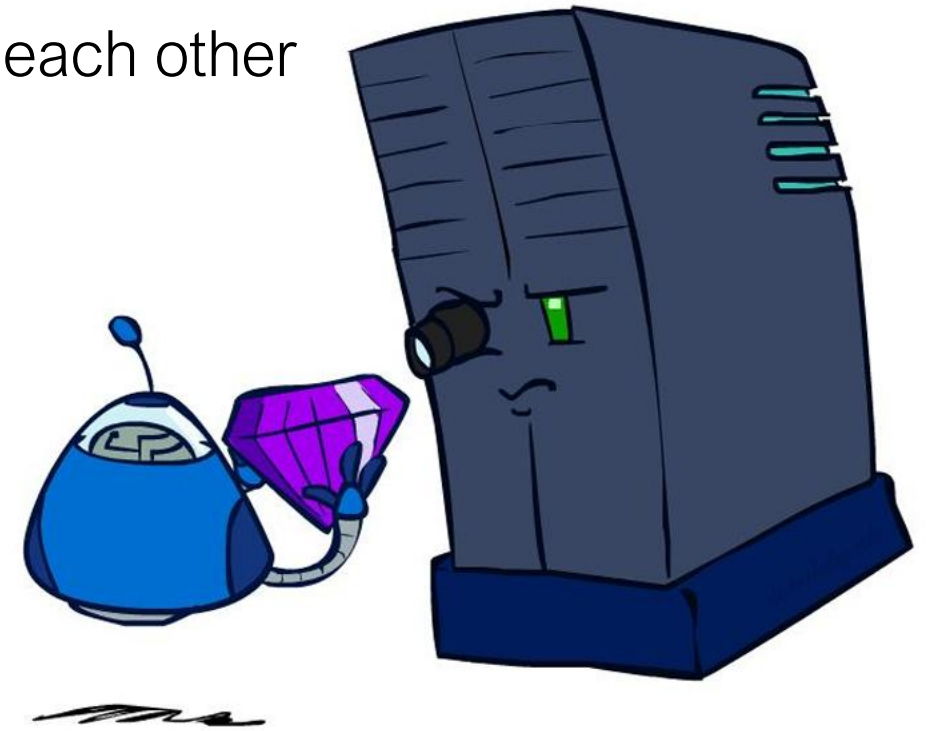


- A danger of “replanning” agents!
  - Pac-Man knows his score goes up if he eats the dot now – (west, then east)
  - He also knows it goes up by exactly the same amount if he eats it later – (east, then west)
  - Within his horizon, there are no other point-scoring chances after the dot
  - Therefore, waiting seems just as good as eating: he may go east, then back west in the next round of replanning!

5. Evaluation Functions

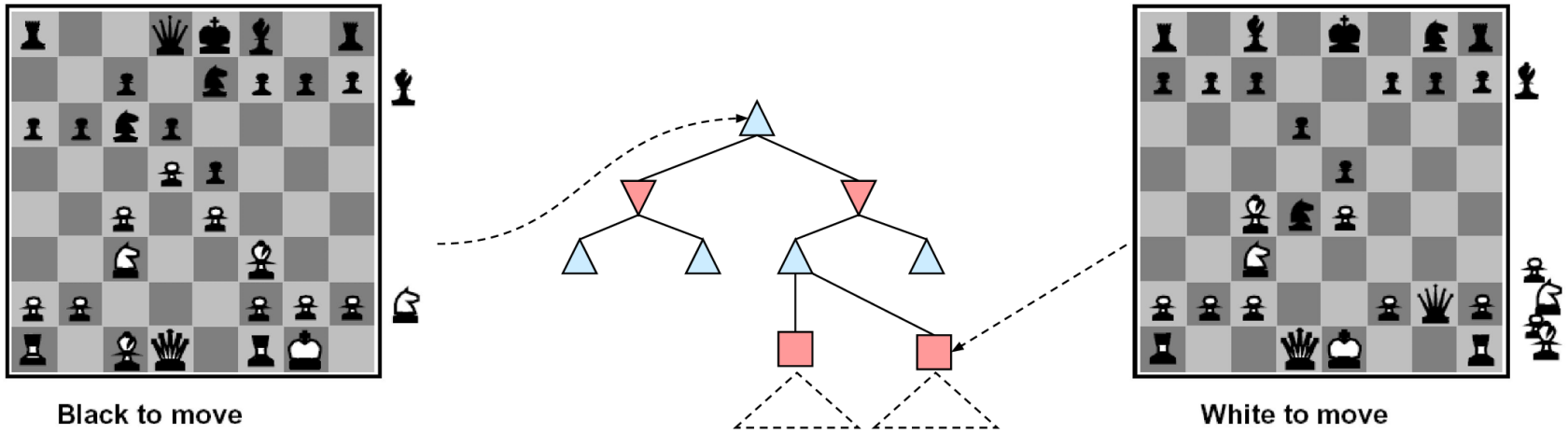
The number you put in when you stop early

- What an evaluation function is trying to be
- The usual form: a weighted sum of features
- Why depth and evaluation quality trade off against each other
- How ordering connects pruning and evaluation



Evaluation Functions

- Evaluation functions score *non-terminals* in depth-limited search



- Ideal function: returns the actual minimax value of the position
- In practice: approximate, e.g. weighted linear sum of features:

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

- e.g. f1(s) = (num white queens – num black queens), etc.